

Project Setup & Configuration

Establishing a clean, modern folder structure is critical for building scalable full-stack applications. By keeping your client-side (frontend) and server-side (backend) logic isolated in dedicated workspaces, you ensure a clean separation of concerns and facilitate smoother deployment pipelines.

1. Creating the Project Folder

To begin, we will organize your local workspace root directory. This acts as the container holding both your frontend application and backend server.

Create a new directory named after your project (e.g., `my-project-folder`) on your system.

Inside that folder, create two new subdirectories:

Client (will house your frontend React & Vite application)

Server (will house your backend Node.js & Express application)

Open the main root folder (e.g., `my-project-folder`) inside your **Visual Studio Code (VS Code)** editor.

Your directory tree should look like this before setup starts:

```
my-project-folder/  <-- Open this in VS Code |— Client/          <-- Frontend Code |— Server/
<-- Backend Code
```

2. Client-Side Setup (Vite + React)

We will configure the client-side of the application using Vite, which offers extremely fast build and development performance for modern React applications.

Step 2.1: Open the Client Folder in Terminal

In VS Code, open a new integrated terminal and navigate into the Client directory:

```
cd Client
```

Step 2.2: Scaffolding with Vite

Execute the following initialization command to build a React application directly in the current directory using Vite's fast template generator:

```
npm create vite@latest . -- --template react
```

Terminal Walkthrough: When prompted, provide the following configuration choices:• Select a framework: **React**• Select a variant: **JavaScript**

Step 2.3: Navigate to Client and Install Packages

If you initialized using the command above (which uses '.' for the current folder), you are already inside the correct location. However, if Vite scaffolded your project into a nested client folder, execute:

```
cd Client
```

Now, execute the package installer command to download React and development dependencies:

```
npm install
```

Step 2.4: Start the Dev Server

Launch the hot-reloading development server with the following command:

```
npm run dev
```

The local development URL will be outputted to your terminal (e.g., <http://localhost:5173>). Open this URL in your web browser to confirm your React interface is successfully active.

3. Server-Side Setup (Node & Express)

Now, we will prepare an organized workspace structure for your API and database configurations inside the backend server workspace.

Step 3.1: Open Server Folder in Terminal

Switch terminal contexts or open a new terminal tab and ensure you are positioned inside the Server directory:

```
cd ../Server
```

Step 3.2: Initialize npm Package

Generate a clean, default package description configuration file by executing:

```
npm init -y
```

Step 3.3: Create Core Files & Directories

To establish clean architecture (such as Model-View-Controller, or MVC design), run terminal creation commands or manually create the following workspace files and folders:

server.js (Responsible for initializing Express, middleware, and starting the web server listener)

models/ (Houses your database schemas/structures, e.g. Mongoose schema definitions)

controllers/ (Contains operational controller functions running backend operations behind paths)

routes/ (Defines endpoints and API paths mapping directly to controller logic)

You can quickly generate this setup with these commands in Unix or Git Bash terminals:

```
touch server.js mkdir models controllers routes
```

4. Final Combined Project Structure

Upon completing these steps, your full workspace folder hierarchy in VS Code is structurally established:

```
my-project-folder/├─ Client/│   │   └─ public/│   │   └─ src/│   │   └─ index.html│   │   └─ package.json│   └─┘├─ vite.config.js├─ Server/│   │   └─ controllers/│   │   └─ models/│   │   └─ routes/│   │   └─ package.json│   └─┘└─ server.js
```